
| | | | | | | |
|---|---|---|---|---|---|---|
| **Student name:** | Leyre Raga Mosteiro | | | | | |
| **Student number:** | 3131025 | | | | | |
| **Faculty:** | Computing Science | | | | | |
| **Course:** | BSCH/BSCO/EXCH | | | Stage/year: | 2 | |
| **Subject:** | Software Development 2 | | | | | |
| **Study Mode:** | Full time | ☑ | | Part-time | | |
| **Lecturer Name:** | Gemma Deery | | | | | |
| **Assignment Title:** | Code Review 3 | | | | | |
| **Date due:** | 30/04/2025 | | | | | |
| **Date submitted:** | 30/04/2025 | | | | | |

Plagiarism disclaimer:

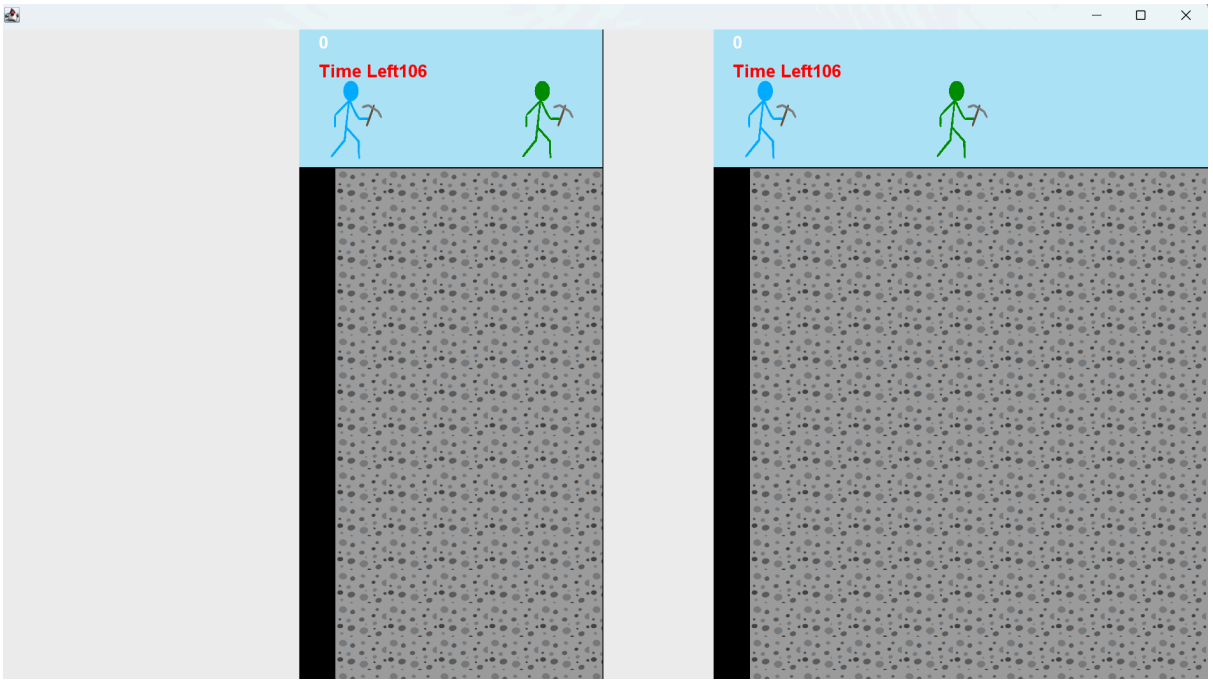
I understand that plagiarism is a serious offence and have read and understood the college policy on plagiarism. I also understand that I may receive a mark of zero if I have not identified and properly attributed sources which have been used, referred to, or have in any way influenced the preparation of this assignment, or if I have knowingly allowed others to plagiarise my work in this way.

I hereby certify that this assignment is my own work, based on my personal study and/or research, and that I have acknowledged all material and sources used in its preparation. I also certify that the assignment has not previously been submitted for assessment and that I have not copied in part or whole or otherwise plagiarised the work of anyone else, including other students.

Signed: _____Leyre_____ Date: _____02/04/2025_____

# Split Screen

The first problem I encountered while programming the split screen is what is happening in the image below:




The goal of the split screen is to divide the singular panel into two equal screens. On each screen, there must be one player (different from the other on the opposite screen), a timer, and a counter for the points. A black line would separate these two screens.

As you can see, the timer and counter for points are printing correctly on both screens. However, two players are on both screens, and you cannot see them in the image, but the camera moves with the player. In addition, there is that white space to the right of the screen.

The two players' problem was easily solved by creating another method identical to the method we had to print the sprites, but with only printing one player in each method.

The white space problem was also easily solved by splitting the screen correctly. I made an error while splitting the screen, so the screen was not split correctly in half like it should be.

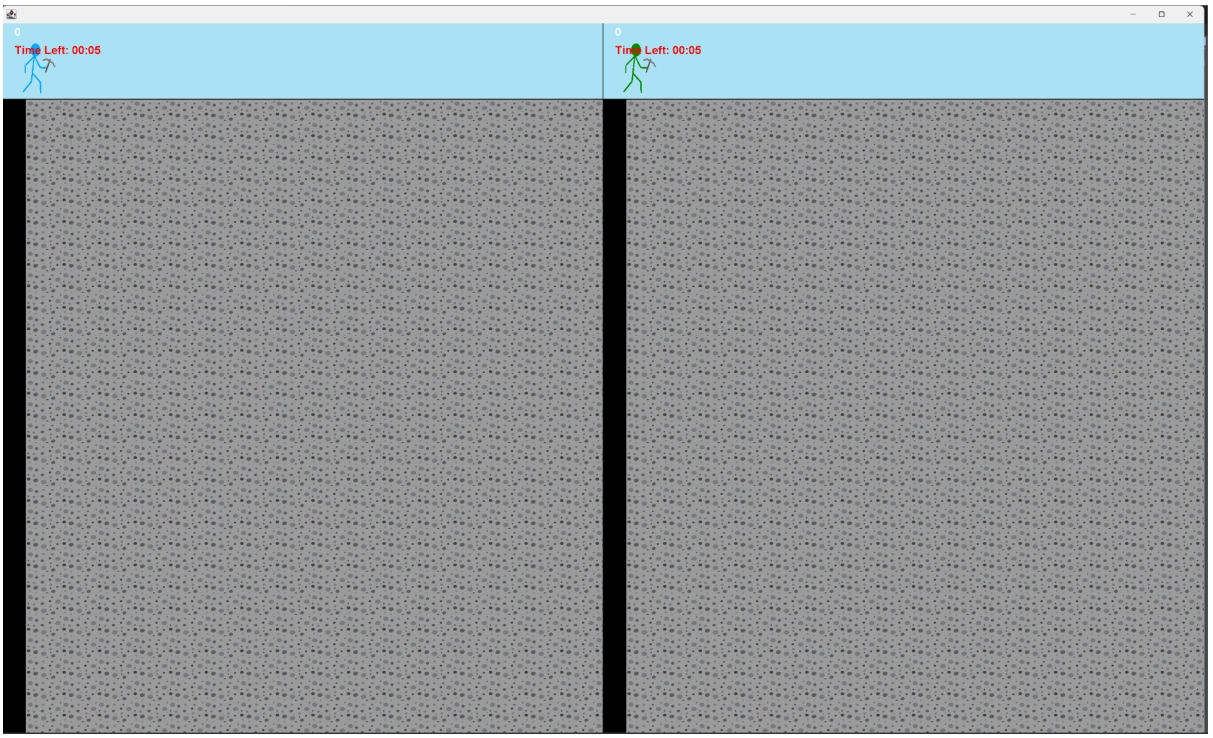
The next problem was not so easily solved and led to many other problems. The camera moved when Player 1 moved and did not move when Player 2 moved, but allowed the player to go out of bounds. For this problem, I needed to clamp the camera, which was easily solved by deleting a line of code that I added, thanks to a video that allowed the camera to move. As previously said, this clamping led to a lot of problems.

# Player 2 Out Of Bounds

Thanks to the clamping, Player 2 was printed out of bounds, meaning that when the screen was displayed, Player 2 seemed not to be painted. I know that it was being printed out of bounds of its screen because I did a debug, where in the console it printed the position of both players and the image it was using.

Once this debugging confirmed that suspicion, I moved to trying to shift the Player 2 so it showed on screen. For that, I just needed to manipulate where the code painted Player 2's sprite by the x-axis. Once it printed correctly, I encountered another problem: Player 2 could not move.

```
//Draw player
graphics.drawImage(player2.getImage(), player2.getX() - (Constants.SCREEN_WIDTH/2), player2.getY(),player2.getWidth(),
```




# Player 2 Not Moving

If not solved, what Player 2 would have experienced was the sprite only being able to jump, and no matter the keys they pressed, the Player would not move except for just moving to jump.

The first thing I checked if it there was anything broken with the player movement. The first thing that I noticed is that the update method did not have an updatePlayerMovement method, so I added it. Unfortunately, this did not solve the problem, so I moved to the next plausible cause of the problem, the KeyListner.

To check if the KeyListener was detecting the keys being pressed for every sprite, I made a debugging tool that printed out the code when a key was pressed.

```
Key Pressed: 68
Key Pressed: 65
```

This is what it showed, and if you research what code binds that key, you will see that 65 is the “A” Key and that 68 is the “D” Key. So the KeyListener was not the thing that was broken, so I moved to my next suspicion. The bounds of Player 2.

I commented the clamp that was in place for Player 2, and once that was out of the way, it confirmed my suspicions Player 2 was moving when you pressed the “A” Key but not when you pressed the “D” Key so that means that bounds of Player 2 where not correctly ensatblished.

The easiest method that I could fix this was by adding bounds to the Player class:

```
//sets movement boundaries
public void setBounds(int left, int right) {
    if(getX() < left) {
        setX(left);
    }

    if (getX() + getWidth() > right) {
        setX(right - getWidth());
    }
}
```

This basically makes sure that the movement is not out of bounds

```
player.setBounds(0, Constants.SCREEN_SIZE.width/2);
player.update();

player2.setBounds(Constants.SCREEN_SIZE.width/2, Constants.SCREEN_SIZE.width);
player2.update();
```

Another thing that was changed on the Player class was how it moved left and right, since I needed to make sure it did not move out of bounds while moving:

```
public void moveRight()
{
    int xMax = Constants.SCREEN_SIZE.width - Constants.PLAYER_WIDTH;
    if(getX() + getWidth() < xMax)
        this.setX(getX() +Constants.PLAYER_SPEED);
}

public void moveLeft()
{
    int xMin = 0;
    if(getX() -Constants.PLAYER_SPEED > xMin) // don't go off screen
        this.setX(getX() -Constants.PLAYER_SPEED);
}
```

After all this was established, Player 2 and Player 1 both moved like they should be moving.

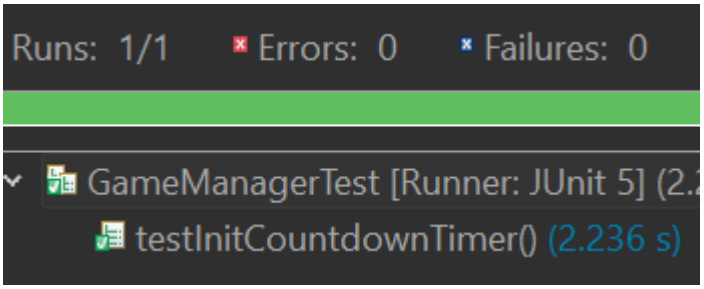
# Countdown Timer

Before, while I was testing, I noticed that the timer, instead of restarting the game when the timer reached 0, continued subtracting and went into the negatives without restarting the game.

This was due to a problem with the logic. I had put what to do in case it reached 0 inside the logic of the timer, when it should have been outside before it entered the logic. This was easily solved by taking the “if” outside and adding a return that stops the updating.

This solved the problem of the countdown timer going into negative numbers, but it still did not solve the problem of not resting, but that will be explained how it was solved in the next part of what I worked on.

Another major change that happened to the countdown timer is how it looked before; it was all in seconds, but now I have changed it so that it is seen in the format minutes: seconds (e.g., 1:45)



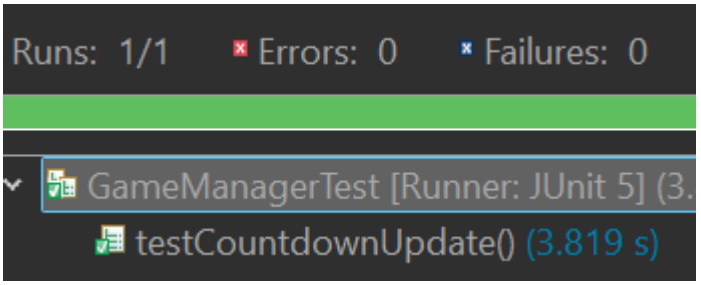

## Restarting Problem

Previously, I added a thread to the code to avoid restarting because once it restarted, the code crashed because it was trying to repaint over the already printed blocks. I needed to look up what the errors meant.

I managed to fix this problem by creating two new arrays that iterate through every time the program is restarted and clears the blocks.

```
//Draw blocks
ArrayList<Block> tempBlocks = new ArrayList<>(blocks); // Make a copy of the blocks list
synchronized(blocks) {
    for (Block block : tempBlocks) {
        graphics.drawImage(block.getImage(), block.getX(), block.getY(), block.getWidth(), block.getHeight(), panel);
    }
}
```

With this problem fixed, it was time to test if the players reset and the timer also reset once the timer reached 0. Once it was tested, the players and the blocks did paint correctly, but what did not print correctly was the timer. It continued to remain 0. This was due to a small error in the logic, where it did not restart correctly due to a line that just needed to be deleted, and after its deletion, the timer did reset.




# Reducing Lag

A problem we still had was that the game was a bit laggy do to it trying to render a lot of blocks at the same time. This was easily solved by changing some things from the variables of the loop that saves the blocks.

This was done by trial and error because if I based it on the screen height and width, and the block height and width, it would paint with blank spaces, since the image, for some reason, no matter how we try to solve it, always has blank spaces.

So this solution was a constant trial and error to see which one would fill the whole screen without printing extra blocks.

```
//saves the position of the blocks in a grid
for(int row = 0; row < rows + 17; row++) {
    for (int column = 0; column < columns + 19; column++) {
        int randIndex = (int) (Math.random()*names.length);
        String fileName = names[randIndex]; //name of the file
        x = column * Constants.BLOCK_WIDTH; //increases the z factor
        y = (Constants.GROUND_HEIGHT + 100) + (row * Constants.BLOCK_HEIGHT); //increases the y factor
        blocks.add(new Blocks(fileName, x, y, Constants.BLOCK_WIDTH, Constants.BLOCK_HEIGHT, blockTypes.get(randIndex).getValue())); //adds the positio
    }
}
```

# Bibliography


- *"List of Java Exceptions."* Programming.Guide, https://programming.guide/java/list-of-java-exceptions.html. Accessed 30 Apr. 2025.
- *"Keys Enum (System.Windows.Forms)."* Microsoft Learn, https://learn.microsoft.com/en-us/dotnet/api/system.windows.forms.keys?view=windowsdesktop-9.0. Accessed 30 Apr. 2025.

- *"How to Create a Split-Screen Display with One Frame and Canvas."* GameDev Stack Exchange, https://gamedev.stackexchange.com/questions/160402/how-to-create-a-split-screen-display-with-one-frame-and-canvas. Accessed 30 Apr. 2025.
- *"Split-Screen for Two Players Java 2D Game Canvas and JFrame."* Stack Overflow, https://stackoverflow.com/questions/58602615/split-screen-for-two-players-java-2d-game-canvas-and-jframe. Accessed 30 Apr. 2025.